

Découvrez ce que cela change pour vous



← Découvrez les librairies Python pour la Data Science

31% de progression

Tracez des graphiques avec Matplotlib



Découvrez Matplotlib

À ce stade, nous avons réuni de nombreuses informations sur nos clients, que nous pourrions mixer et représenter au sein de différents graphiques.

Par exemple, plutôt que d'avoir l'information sous forme de tableau, il serait intéressant de représenter le chiffre d'affaires total par agence. On peut aussi regarder le taux d'endettement de nos clients en fonction de leur revenu, pour voir si une tendance se dégage.

Il nous faut donc une librairie pour réaliser ces différents graphiques avec Python. Il en existe une multitude, et c'est parfois un peu compliqué de comprendre l'intérêt et la plus-value de l'une par rapport à une autre. C'est pourquoi je vous propose de regarder en détail la librairie principalement utilisée pour réaliser des visualisations : **Matplotlib**. Enfin, nous allons plutôt utiliser **pyplot** qui est inclus dans Matplotlib.

Voilà comment importer cette dernière :

```
1 import matplotlib.pyplot as plt
```

Si dans votre vie vous avez été amené à réaliser des calculs scientifiques, ce nom vous dit peut être quelque chose. C'est normal ! Car la façon de créer des graphiques avec Matplotlib est calquée sur celle d'un autre langage très utilisé en calcul scientifique : **MATLAB**. Si vous avez déjà utilisé ce dernier, vous allez retrouver de grandes similitudes.

Chaque représentation graphique a une fonction correspondante avec Matplotlib :

- nuage de points ou *scatter plot*, en anglais : `scatter()` ;
- diagrammes en ligne ou en courbes : `plot()` ;
- diagrammes en barres : `bar()` ;
- histogrammes : `hist()` ;
- diagrammes circulaires : `pie()` .

Que diriez-vous de découvrir une application concrète de ces fonctions ? C'est parti !

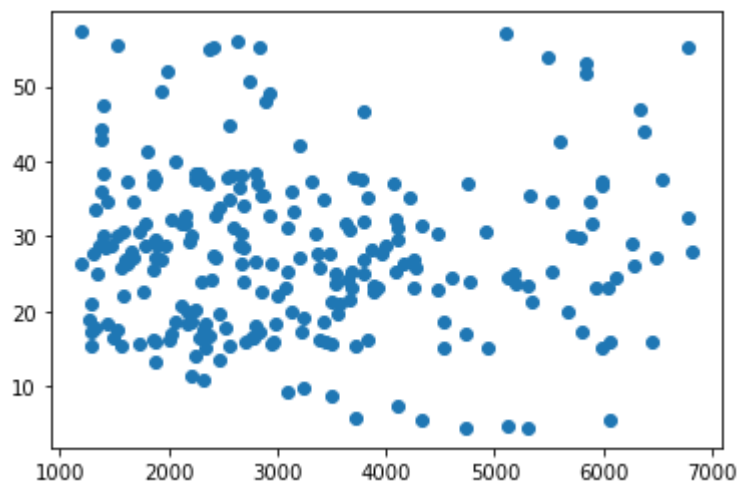
Tracez vos premiers graphiques

Le nuage de points

Cherchons à représenter un des graphiques cités ci-dessus : représenter le taux d'endettement en fonction du revenu. Ces deux variables sont numériques, sans évolution dans le temps : nous allons donc tracer un nuage de points, via la fonction `scatter` . Cette fonction nécessite de définir x et y, qui sont les valeurs à placer en abscisse et en ordonnée :

```
1 plt.scatter(prets['revenu'], prets['taux_endettement'])
```

... pour obtenir le résultat suivant :



Application de la fonction scatter

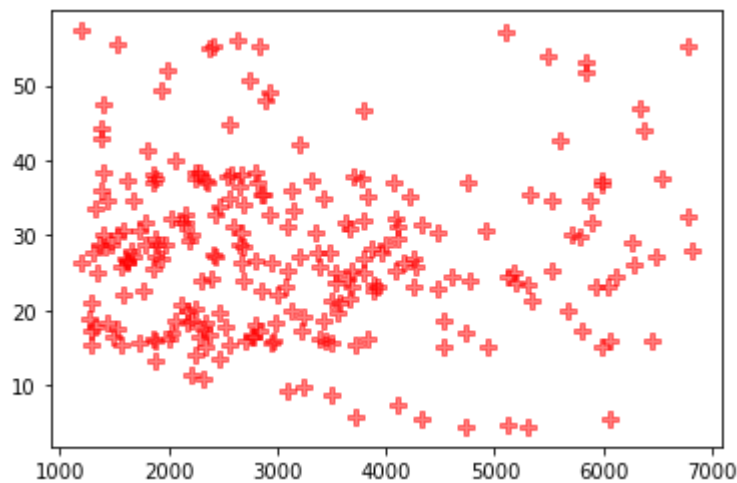
Il existe de nombreuses options pour personnaliser un nuage de points. On peut modifier :

- la couleur des points, en utilisant l'argument `color` ou `c` ;
- la taille des points, via l'argument `size` ou `s` ;
- le type de marqueur via l'argument `marker` ;
- la transparence des points via l'argument `alpha` .

Reprenons le même graphique, avec des croix rouges dont la taille aura été modifiée, et avec 50 % de transparence :

```
1 plt.scatter(prets['revenu'], prets['taux_endettement'],  
2            s=60, alpha=0.5, c='red', marker='P')
```

... et la visualisation correspondante :



Application d'options sur la fonction scatter

Cette liste n'est qu'une infime partie de ce que vous pouvez modifier... Vous trouverez la liste complète des arguments sur [la documentation officielle de la fonction](#).

On souhaite maintenant représenter le chiffre d'affaires total par agence. Comme nous l'avons vu précédemment, il existe deux possibilités pour cela : soit un diagramme à barres, soit un diagramme circulaire. Nous allons réaliser ces deux visualisations.

Le diagramme circulaire

Là où nous autres francophones et amateurs de fromages voyons dans le diagramme circulaire un camembert, les anglo-saxons y voient plutôt une tarte (*pie*, en anglais) à découper en plusieurs parts ! C'est de là que provient le nom de la fonction permettant de tracer un diagramme circulaire : `pie` .

Son utilisation est très similaire à celle de `scatter` . Il y a deux arguments à préciser : `labels` , correspondant à la variable non numérique, celle sur laquelle ont été agrégées les données, et `x`, les valeurs agrégées correspondantes.

La première étape va donc être d'agréger les données :

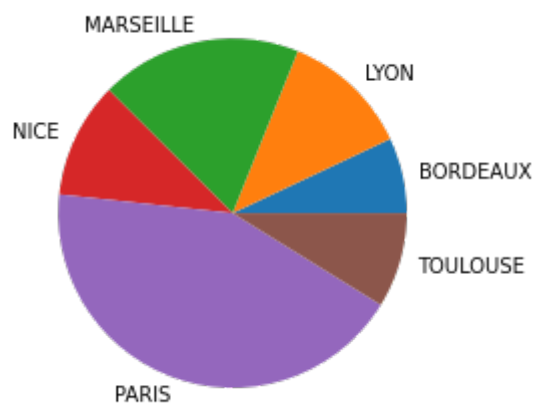
```
1 data = prets.groupby('ville')['remboursement'].sum()
2 data = data.reset_index()
```

Le `reset_index` est nécessaire car nous allons avoir besoin de la variable `ville` dans la création de nos prochains graphiques.

Traçons à présent notre *camembert* à partir de ces données agrégées :

```
1 plt.pie(x=data['remboursement'], labels=data['ville'])
```

... pour avoir le résultat suivant :



Application de la fonction pie

On peut également améliorer ce graphique en affichant textuellement le pourcentage associé à chaque "part". Pour cela, il faut spécifier un format numérique via l'argument `autopct` . Par exemple :

```
1 plt.pie(x=data['remboursement'], labels=data['ville'], autopct='%0.2f%%')
```

Explicitons rapidement ce format : il permet d'afficher la part du chiffre d'affaires total réalisé par chaque agence, avec 2 chiffres après la virgule, et suivie du caractère `%` .

Vous pourrez trouver tous les arguments possibles de la fonction `pie` [sur la documentation officielle](#).

Le diagramme à barres

L'alternative au diagramme circulaire est un diagramme à barres ! On peut représenter exactement la même information, mais avec une perspective différente.

Pour utiliser la fonction `bar` , qui est l'implémentation de Matplotlib des diagrammes à

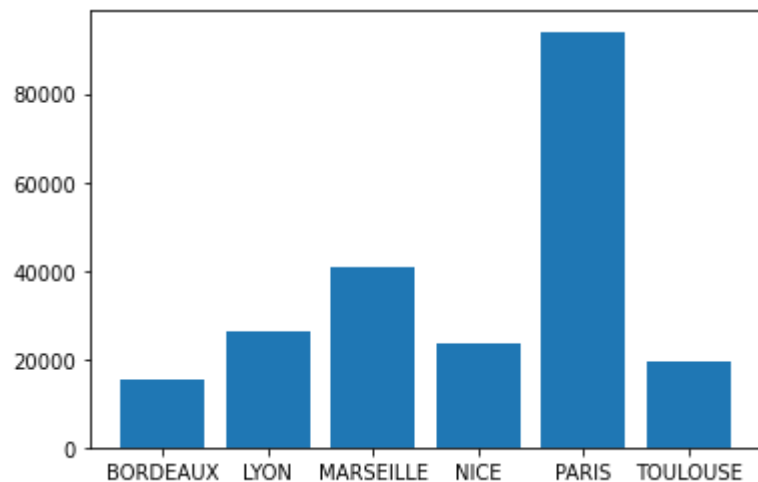
barres, il faut préciser deux arguments :

- `x` : les différentes valeurs de la variable non numérique, l'équivalent du `labels` de `pie` ;
- `height` : les valeurs agrégées, équivalent du `x` de `pie` .

Représentons maintenant la même information – le chiffre d'affaires total par agence – via un diagramme à barres :

```
1 plt.bar(height=data['remboursement'], x=data['ville'])
```

... pour obtenir :

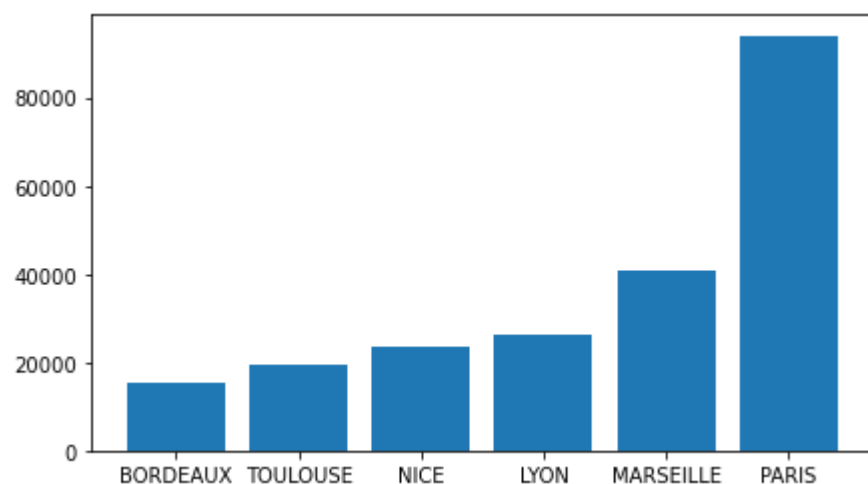


Application de la fonction bar

Et si vous souhaitez ordonner du plus grand au plus petit ? Il faudra *tricher* un petit peu. En effet, la fonction `bar` de Matplotlib ne permet pas, par défaut, de réaliser cela... MAIS on peut tout de même trier le data frame en amont.

```
1 data_sorted = data.sort_values('remboursement', ascending=False)  
2 plt.bar(height=data_sorted['remboursement'], x=data_sorted['ville'])
```

Un peu plus sympa comme ça, non ?



Comme d'habitude, je vous redirige vers [la documentation de la fonction](#) si vous souhaitez approfondir cette dernière !

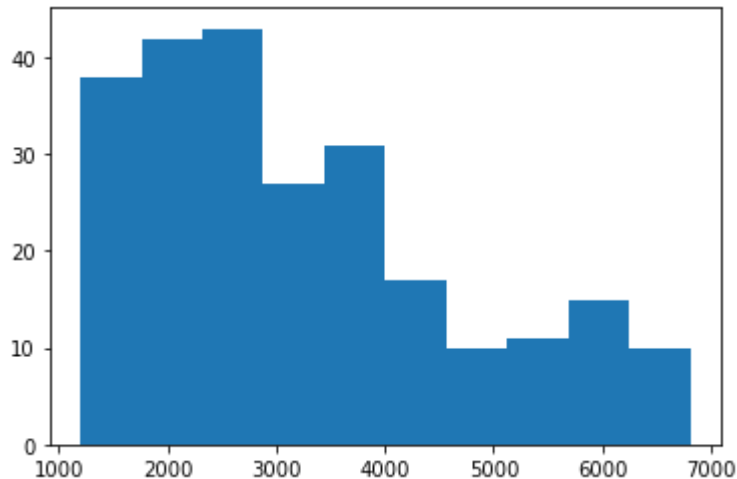
L'histogramme

Lorsqu'il s'agit de connaissance client dans le milieu bancaire, on arrive assez rapidement à la question : Comment se répartissent nos clients en termes de revenus ? A-t-on majoritairement des classes moyennes ? ou des revenus plus modestes ?

L'histogramme est particulièrement utile lorsqu'on souhaite avoir une idée de la distribution d'une variable, et donc particulièrement adéquat pour répondre à cette problématique. La fonction Matplotlib correspondante est `hist` . Il suffit de lui passer en paramètre la variable numérique dont on souhaite connaître la distribution :

```
1 plt.hist(prets['revenu'])
```

On obtient ainsi :



Application de la fonction hist

En un coup d'œil, on peut déterminer, grâce à l'histogramme, que nous avons majoritairement des revenus modestes au sein de notre clientèle.

Il existe de nombreuses possibilités de personnalisation, notamment sur le nombre de tranches, leur composition, etc. Vous pourrez retrouver tout cela, une fois de plus, dans [la documentation de la fonction](#) .

Les courbes

On souhaite suivre l'évolution du chiffre d'affaires sur les 4 premiers mois, pour voir la façon dont il varie, et éventuellement anticiper la suite.

Nous avons pour cela à notre disposition le chiffre d'affaires réalisé par notre banque sur les prêts de janvier à avril 2013 :

```
evolution_ca = pd.DataFrame({'date': ['2013-01-01', '2013-02-01',  
                                     '2013-03-01', '2013-04-01'],  
                             "chiffre d'affaire": [183000, 193020,  
                                                  179032, 219174]})  
evolution_ca
```

	date	chiffre d'affaire
0	2013-01-01	183000
1	2013-02-01	193020
2	2013-03-01	179032
3	2013-04-01	219174

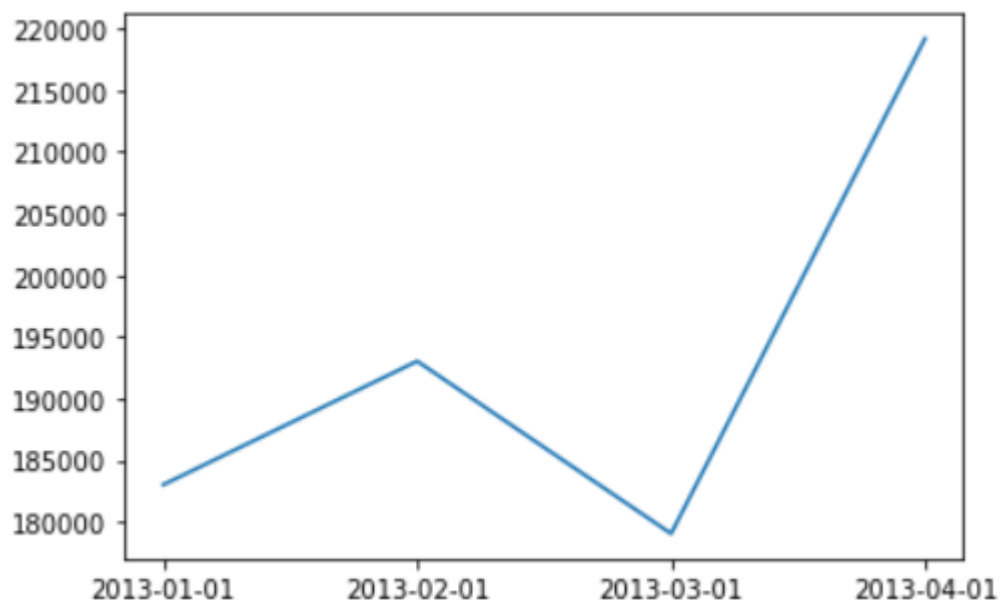
Data frame à utiliser pour tracer une courbe

On a ici une évolution dans le temps, donc le choix de représentation le plus pertinent serait de tracer une courbe.

Pour ce faire, on utilise la fonction `plot` de Matplotlib. Celle-ci prend deux arguments en entrée : les informations à mettre en abscisse, et celles à mettre en ordonnée. Voilà comment tracer l'évolution de notre chiffre d'affaires :

```
1 plt.plot(evolution_ca['date'], evolution_ca["chiffre d'affaire"])
```

Nous avons eu un chiffre d'affaires en hausse globale sur le premier tiers de 2013 :



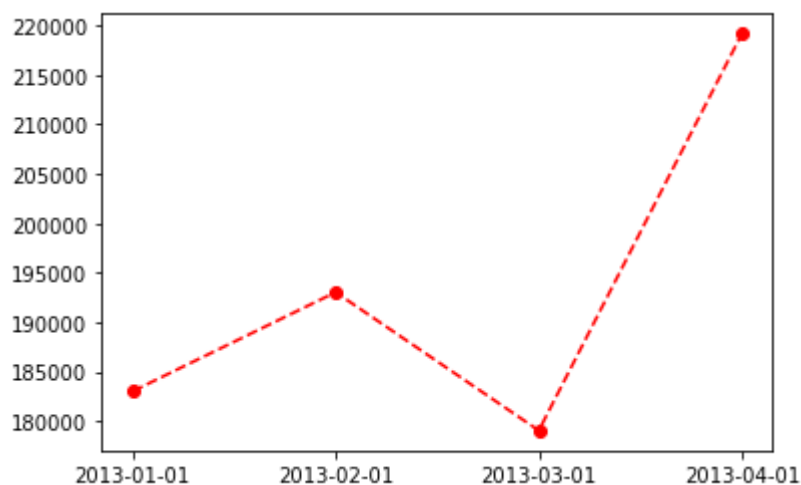
Application de la fonction plot

Comme pour le nuage de points, il existe de très nombreuses options de personnalisation. Vous pouvez jouer sur la couleur de la ligne (`color` ou `c`), son style (`linestyle` ou `ls`), son épaisseur (`linewidth` ou `lw`), si on souhaite ajouter un marqueur en plus de la ligne (`marker`), etc. Vous trouverez l'ensemble des possibilités sur [la documentation de Matplotlib](#).

Voici par exemple le même graphique, avec des lignes rouges hachurées, où on ajoute un point à chaque date :

```
1 plt.plot(evolution_ca['date'], evolution_ca["chiffre d'affaire"],
2 marker='o', linestyle='--', color='red')
```

Ça donne un autre style à notre graphique :



Application d'options sur la fonction plot

Ces différents graphiques sont sympas, mais ils ne respectent pas vraiment les bonnes pratiques présentées précédemment...

C'est un excellent point ! En effet, il manque pas mal d'informations sur l'ensemble de ces graphiques... mais ne vous inquiétez pas, ce n'est pas un oubli de ma part, nous aurons l'occasion de voir comment corriger tout cela dans le prochain chapitre.

Créez plusieurs graphiques sur une même fenêtre

Nous avons 6 agences différentes avec plusieurs dizaines de clients par agence. Le responsable national souhaite avoir, en un seul graphique, une idée du comportement des agences en termes d'attribution de taux, en fonction du revenu.

Représenter le revenu en fonction du taux est assez simple : on peut faire cela via un nuage de points. Mais comment représenter, en plus, l'information de l'agence ?

En ajoutant une information supplémentaire à notre graphique, comme la couleur des points !

Cependant, il n'existe pas d'option par défaut avec Matplotlib pour mettre en couleur les points. On va être forcés de créer plusieurs graphiques – un par agence – qu'on va superposer sur une seule et même fenêtre graphique.

Laissez-moi vous montrer en vidéo comment réaliser cela :

03:07

Maintenant que vous savez tracer plusieurs graphiques dans une même fenêtre, je vous propose de mettre en pratique les connaissances acquises dans ce chapitre !

À vous de jouer



Contexte

Vous êtes en train de préparer le reporting mensuel à présenter chaque fin de mois à votre responsable. La présentation devra comprendre certains graphiques clés : vous allez donc devoir utiliser vos compétences en data visualisation pour produire les différentes visualisations attendues.

Consignes

Les graphiques à produire sont donc :

- la proportion de chaque type de prêt ;
- le bénéfice mensuel réalisé en fonction du revenu du client pour les prêts immobiliers ;
- la distribution des bénéfices réalisés ;
- le bénéfice mensuel total réalisé par agence.

Je vous invite à vous entraîner en vous rendant sur l'[exercice](#).

Vérifiez votre travail

Bien joué ! Voici [la correction correspondante](#).

En résumé

- Matplotlib met à disposition une fonction par type de graphique souhaité :
 - `plot` : pour les courbes ;
 - `bar` : pour les diagrammes à barres ;
 - `pie` : pour les diagrammes circulaires ;
 - `hist` : pour faire des histogrammes ;
 - `scatter` : pour les nuages de points.
- Personnalisez vos différents graphiques en utilisant les différents paramètres de chaque fonction.
- Tracez différents graphiques sur une seule fenêtre graphique, pour ajouter des dimensions supplémentaires.

Je vous propose maintenant de plonger un peu plus en profondeur dans les possibilités offertes par Matplotlib pour personnaliser un graphique.

 [Avez-vous une suggestion pour nous ?](#) 